



CARAGA STATE UNIVERSITY
COLLEGE OF COMPUTING AND INFORMATION SCIENCES
CSU-Main Campus, Ampayon, Butuan City, Philippines
Competence Service Uprightness
YOUR BEST OPTION TO SUCCESS



PREDICTIVE MODELING FOR CUSTOMER CHURN: A COMPARATIVE ANALYSIS OF KNN, SVM, AND ANN

MEMBERS OF THE GROUP:

TRISTAN RHYL C. PENASO

AIRAH NICHOLE MONTILLANO

KYLLE MAE MERCADO

COURSE: IS 108 – BUSINESS INTELLIGENCE FINAL PROJECT

SY 2025-2026

1. Business Problem Addressed

This project addresses the critical business problem of Customer Churn Prediction. In the telecommunications sector, customer retention is vastly more cost-effective than customer acquisition. The goal of this predictive modeling application is to identify patterns and predict which customers are at high risk of canceling their subscriptions. By accurately classifying these at-risk customers, the business can proactively deploy targeted retention strategies, such as promotional discounts or personalized customer service, thereby reducing revenue loss.

2. Dataset Used

- The application utilizes the [WA_Fn-UseC_-Telco-Customer-Churn.csv](#) dataset to train and evaluate the predictive models. This dataset was explicitly selected because it accurately simulates a real-world Business Intelligence scenario, providing a comprehensive mix of numerical and categorical variables (such as tenure, monthly charges, and contract types) that are ideal for testing the pattern-recognition capabilities of KNN, SVM, and ANN algorithms.
- The raw dataset contains 7,043 customer records, capturing demographic details, account information, and subscribed services. The target variable is **Churn**, which indicates whether the customer left the company within the last month, serving as a highly realistic metric for evaluating business retention strategies.

3. Data Preprocessing Steps

To ensure the models receive clean and mathematically sound data, the application executes a dynamic preprocessing pipeline. The following steps were implemented:

Data Cleaning: Blank strings are automatically identified and converted to NaN values to prevent processing errors. In this dataset, 11 rows containing missing values were dropped, resulting in a clean dataset of 7,032 rows for training and testing.

High-Cardinality Column Removal: Prior to encoding, categorical columns where the number of unique values exceeds 50% of total rows are automatically identified and removed. Columns meeting this threshold (e.g., customerID) are identifier fields that carry no predictive signal — one-hot encoding them would generate thousands of dummy columns, inflating feature space, corrupting StandardScaler's fit, and making correlation and distance computations impractically slow. The pipeline logs all dropped columns transparently in the application interface.

Numeric Coercion: Columns stored as strings that contain primarily numeric data (such as [TotalCharges](#)) are forcefully converted to numeric data types before missing value imputation.

Handling Missing Values: Missing values are handled dynamically based on user input, with options to drop rows entirely or impute them (using the mean for numeric columns and the mode for categorical columns).

Label Encoding: Categorical columns are encoded using a two-strategy approach based on cardinality. Binary columns with only 2 unique values (e.g., gender, PhoneService) are safely label-encoded to 0/1 integers, as no false ordinality is introduced. Nominal columns with 3 or more unordered categories (e.g., InternetService, Contract, PaymentMethod) are one-hot encoded using `get_dummies`, producing separate binary columns per category. This prevents distance-based models like KNN from computing meaningless ordinal distances between unrelated categories. The target variable (Churn) is separately encoded to integers to satisfy MLPClassifier's strict input requirements.

Feature Scaling: To ensure features with larger numerical ranges do not dominate the distance metrics in KNN or margin calculations in SVM, `StandardScaler` is applied to achieve zero-mean and unit-variance.

Train/Test Split: The dataset is split into training and testing sets (80/20) using stratification to preserve the exact class distribution of the target variable in both sets.

Feature Selection: A correlation heatmap is generated to evaluate the relationship between features and the target variable, allowing users to filter out noisy or low-correlation data prior to training (**See Figure 1: Correlation Heatmap**). The feature importance bar chart is anchored at zero on the x-axis to ensure bar lengths are proportionally comparable across all features regardless of their minimum correlation value.

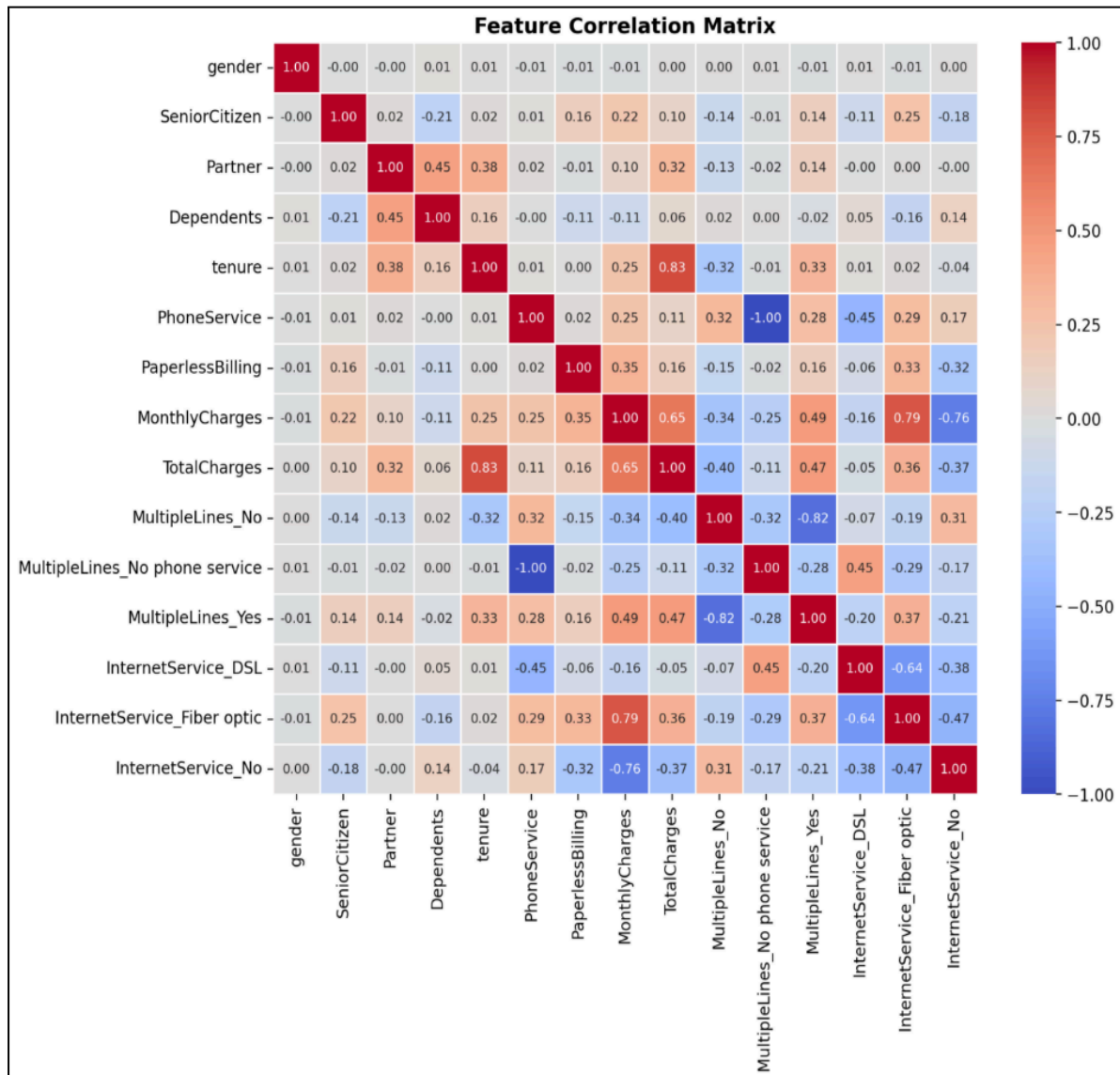


Figure 1: Correlation Heatmap

4. Application Functionality & User Interface

The predictive modeling pipeline is wrapped in a dynamic, interactive web application built with Streamlit. Instead of hardcoded data processing, the application features a functional User Interface (UI) that allows end-users to upload new datasets, dynamically toggle preprocessing settings (like missing value imputation and scaling), and select specific target variables. Furthermore, the application includes a "Live Prediction" module where users can input individual customer metrics to receive an instant, majority-vote churn prediction from all three trained models. Furthermore, Phase 4 features a live prediction module where users input individual customer metrics and receive an instant majority-vote churn prediction from all three trained models simultaneously (See Figure 2: Phase 4 - Live Prediction Output).

Live Prediction Engine

Enter real feature values below and click Predict to get a live prediction from all three trained models.

SeniorCitizen 0.1624	Partner 0.4825	Dependents 0.2985	tenure 32.4218
PaperlessBilling 0.5927	MonthlyCharges 64.7982	TotalCharges 2283.3004	InternetService_DSL 0.3436
InternetService_Fiber optic 0.4403	InternetService_No 0.2162	OnlineSecurity_No 0.4973	OnlineSecurity_No internet service 0.2162
OnlineSecurity_Yes 0.2865	OnlineBackup_No 0.4390	OnlineBackup_No internet service 0.2162	OnlineBackup_Yes 0.3449
DeviceProtection_No 0.4400	DeviceProtection_No internet service 0.2162	DeviceProtection_Yes 0.3439	TechSupport_No 0.4937
TechSupport_No internet service 0.2162	TechSupport_Yes 0.2901	StreamingTV_No 0.3995	StreamingTV_No internet service 0.2162
StreamingTV_Yes 0.3844	StreamingMovies_No 0.3955	StreamingMovies_No internet service 0.2162	StreamingMovies_Yes 0.3884
Contract_Month-to-month 0.5511	Contract_One year 0.2093	Contract_Two year 0.2396	PaymentMethod_Bank transfer (automatic) 0.2193
PaymentMethod_Credit card (automatic) 0.2163	PaymentMethod_Electronic check 0.3363	PaymentMethod_Mailed check 0.2281	

Predict

Prediction Results

KNN Prediction No	SVM Prediction No	ANN Prediction No
-----------------------------	-----------------------------	-----------------------------

All 3 models agree: No — High confidence prediction.

Figure 2: Phase 4 - Live Prediction Output

5. Description of Algorithm Implementation

The application successfully implements and compares three supervised learning models:

5.1 K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based learner; it builds no explicit model during training. At prediction time, it locates the k training examples closest to the new input point and assigns the majority class among those neighbors. The entire training set is retained in memory and searched at every inference call.

Distance is computed using the Minkowski metric, which at $p = 2$ reduces to the standard Euclidean distance:

$$d(x, x') = \sqrt[p]{\sum_i (x_i - x'_i)^p}$$

This formula measures the straight-line distance between two points in n-dimensional feature space. Because distance is computed directly from raw feature values, StandardScaler is not optional; without it, a feature like TotalCharges (range ~0-9,000) would mathematically dominate Tenure (range 1-72), rendering the distance meaningless. For a new input, the algorithm ranks all training records by distance, selects the 5 nearest neighbors, and assigns the class appearing most frequently among them: $\text{argmax}_c |\{i : y_i = c\}|$.

Configuration: $k = 5$ (tunable; GridSearchCV tests $k \in \{3, 5, 7, 9, 11, 15\}$), uniform weighting, Minkowski metric.

Limitation: All computation is deferred to inference time, $O(n \cdot d)$ per query. Performance degrades significantly on high-dimensional data (curse of dimensionality).

5.2 Support Vector Machine (SVM)

SVM is a discriminative classifier that finds the hyperplane maximising the *margin*; the perpendicular distance between the decision boundary and the nearest training examples of each class (the support vectors). Only these boundary-adjacent points influence the final model, making SVM robust to outliers far from the margin.

The separating hyperplane is defined as:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

where $\mathbf{w}$ is the weight vector normal to the hyperplane and b is the bias scalar. For non-linearly separable data, the soft-margin formulation introduces slack variables ξ_i to permit controlled misclassification:

$$\text{minimise } \frac{1}{2} \|\mathbf{w}\|^2 + C \cdot \sum_i \xi_i \text{ subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

The regularisation parameter C controls the bias-variance trade-off: small C allows a wider margin with more misclassifications; large C penalises every error more heavily at the cost of a narrower margin.

Because customer churn data is not linearly separable in the original feature space, our implementation uses the Radial Basis Function (RBF) kernel, which implicitly maps inputs into an infinite-dimensional Hilbert space where a linear boundary becomes achievable:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

The kernel is evaluated during training without ever explicitly computing the high-dimensional mapping; only pairwise inner products are required.

Configuration: kernel = RBF, C = 1.0, class_weight = 'balanced' (reweights the loss for minority-class samples), probability = True (Platt scaling for confidence outputs).

Limitation: Training complexity is $O(n^2)$ to $O(n^3)$ in n ; kernel choice and C require careful tuning.

5.3 Artificial Neural Network - Multi-Layer Perceptron (ANN / MLP)

The ANN is a feed-forward Multi-Layer Perceptron with one or more hidden layers. Information flows from input features through each hidden layer via learned weight matrices, with a non-linear activation applied at every neuron. This allows the network to approximate arbitrarily complex decision boundaries unlike KNN's local averaging or SVM's single kernel-mapped hyperplane.

Forward pass (single hidden layer):

$$h = \text{ReLU}(W_1 \cdot x + b_1) \text{ [hidden layer]} \quad \hat{y} = \text{softmax}(W_2 \cdot h + b_2) \text{ [output layer]}$$

ReLU activation - $f(z) = \max(0, z)$ was chosen over sigmoid or tanh because it does not saturate for positive values, mitigating the vanishing gradient problem that makes deeper networks difficult to train.

Training via backpropagation: The network minimises cross-entropy loss L by computing the gradient of L with respect to every weight using the chain rule, propagating error signals backwards from the output layer to the input layer:

$$\partial L / \partial w = (\partial L / \partial a)(\partial a / \partial z)(\partial z / \partial w)$$

The **Adam optimiser** (Kingma & Ba, 2015) adapts the learning rate per-parameter using exponentially decaying estimates of the first moment (mean gradient) $m_\square$ and second moment (uncentered variance) $v_\square$:

$$m_\square = \beta_1 \cdot m_{\square-1} + (1 - \beta_1) \cdot g_\square \quad v_\square = \beta_2 \cdot v_{\square-1} + (1 - \beta_2) \cdot g_\square^2 \quad w = w - \alpha \cdot m_\square / (\sqrt{v_\square} + \epsilon)$$

This per-parameter adaptation makes Adam more robust than vanilla SGD on sparse or noisy gradients. Early stopping monitors a held-out 10% validation split and halts training once validation loss stops improving, preventing overfitting without manual epoch tuning.

Configuration: hidden_layer_sizes = (100,), activation = ReLU, solver = Adam, max_iter = 500, early_stopping = True.

Limitation: Requires more data to generalise than KNN or SVM; sensitive to initialisation and hyperparameters; predictions are harder to interpret (black box).

6. Evaluation Metrics Used

To evaluate and compare model performance on the unseen test set, the application computes the following required metrics:

- **Accuracy:** The overall percentage of correct predictions.
- **Precision (Macro):** The ratio of true positive predictions to the total predicted positives, averaged equally across all classes.
- **Recall (Macro):** The ratio of true positives to the actual total positives.
- **F1-Score (Macro):** The harmonic mean of Precision and Recall, which is crucial for imbalanced datasets like customer churn.
- **Confusion Matrix:** A visual Seaborn heatmap displaying the exact counts of True Positives, True Negatives, False Positives, and False Negatives for each algorithm.
- **Cross-Validation (5-Fold):** Applied to the training set to provide a more robust, generalized estimate of model accuracy across multiple data splits, reducing the risk of overfitting.

7. Comparison of Results

After processing the dataset, filtering out low-correlation features, and executing an 80/20 train/test split (Train: 5,625 rows | Test: 1,407 rows), the models yielded the following performance metrics:

Cross-Validation (5-Fold) Accuracy Results:

- **ANN:** 79.41%
- **KNN:** 75.09%
- **SVM:** 74.49%

Test Set Performance Metrics:

	Accuracy	Precision	Recall	F1-Score
ANN Performance	79.10%	73.20%	71.78%	72.41%
KNN Performance	74.56%	67.35%	67.23%	67.29%
SVM Performance	72.42%	72.42%	73.80%	69.36%

Analysis: The Artificial Neural Network (ANN) is the most effective model for this specific business problem. In churn prediction, datasets are naturally imbalanced (most customers stay, a smaller fraction leave), meaning overall accuracy alone can be misleading. However, the ANN achieved the

highest performance across almost every critical metric, including overall Accuracy (79.10%), Macro F1-Score (72.41%), and Macro Precision (73.20%).

Validating this with the visual outputs (See **Figure 3: Confusion Matrices**), we observe a critical business trade-off. While the SVM identified more raw churners (287 True Positives versus the ANN's 210), it did so at the heavy cost of 301 False Positives, flagging 301 loyal, non-churning customers as flight risks. This yields an SVM churn-class Precision of approximately 48.8% ($287 \div 588$ predicted churners), meaning over half of all customers the SVM targets for retention intervention are actually false alarms.

By contrast, the ANN produced only 130 False Positives, resulting in a much stronger churn-class Precision of approximately 61.8% ($210 \div 340$ predicted churners). For the business, the ANN's retention flags are far more actionable. It significantly reduces wasted intervention spend (such as unnecessary promotional discounts) while still successfully catching the majority of genuine churners.

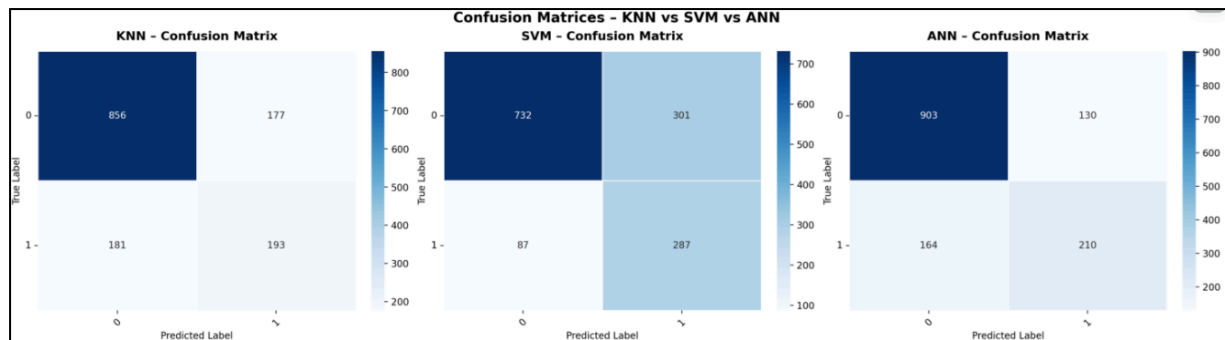


Figure 3: Confusion Matrices

8. Conclusion and Recommendations

Conclusion: This application successfully demonstrates a complete, end-to-end predictive modeling workflow. By analyzing customer behavior, we determined that machine learning models, specifically the Artificial Neural Network (ANN) can accurately predict telecom customer churn based on historical account data. The ANN proved to be the superior model by prioritizing a realistic balance of precision and recall, ensuring that the identification of at-risk customers translates into cost-effective business action rather than expensive false alarms.

Business Recommendations:

- **Model Deployment:** The telecommunications company should deploy the ANN model to score active customers on a monthly basis. Customers flagged by the model as a churn risk should be immediately routed to the proactive retention team.



CARAGA STATE UNIVERSITY
COLLEGE OF COMPUTING AND INFORMATION SCIENCES
CSU-Main Campus, Ampayon, Butuan City, Philippines
Competence Service Uprightness
YOUR BEST OPTION TO SUCCESS



- **Targeted Intervention:** Instead of offering blanket discounts to all customers which unnecessarily degrades overall revenue, the company can preserve capital by offering targeted incentives exclusively to the high-probability segment identified by the ANN.
- **Strategic Feature Focus:** Based on our feature selection and correlation analysis, factors such as Contract Type (specifically Month-to-month contracts) and Tenure are the strongest predictors of churn. The business should consider restructuring its pricing models to heavily incentivize customers to transition from month-to-month plans into long-term contracts early in their lifecycle to improve baseline retention.

References

- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/BF00994018>
- Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27. <https://doi.org/10.1109/TIT.1967.1053964>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Rio, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2), 251–257. [https://doi.org/10.1016/0893-6080\(91\)90009-T](https://doi.org/10.1016/0893-6080(91)90009-T)
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- IBM. (n.d.). *Telco customer churn* [Dataset]. Kaggle. <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015)*. <https://arxiv.org/abs/1412.6980>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 445, 51–56. <https://doi.org/10.25080/Majora-92bf1922-00a>
- Pearson, K. (1895). Note on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*, 58, 240–242. <https://doi.org/10.1098/rspl.1895.0041>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536. <https://doi.org/10.1038/323533a0>



CARAGA STATE UNIVERSITY
COLLEGE OF COMPUTING AND INFORMATION SCIENCES
CSU-Main Campus, Ampayon, Butuan City, Philippines
Competence Service Uprightness
YOUR BEST OPTION TO SUCCESS



Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427–437.
<https://doi.org/10.1016/j.ipm.2009.03.002>

Streamlit Inc. (2024). *Streamlit documentation*. <https://docs.streamlit.io>

Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>